

MİMARİ RAPORU

Araç Takip Sistemi

Vehicle Tracking Simulation — olay güdümlü mikroservis mimarisi

105 araçlık bir filonun konumunu saniyede üreten, Kafka üzerinden işleyen, kural motorlarından geçirip ihlal üreten ve canlı haritaya basan uçtan uca bir telemetri hattı. Bu rapor sistemin **neden** bu şekilde kurulduğunu anlatır — sadece neyden oluştuğunu değil.

PLATFORM

Java 21 · Spring Boot 3.3.5

MODÜLLER

8 Maven modülü

VERİ

TimescaleDB · 22 migration

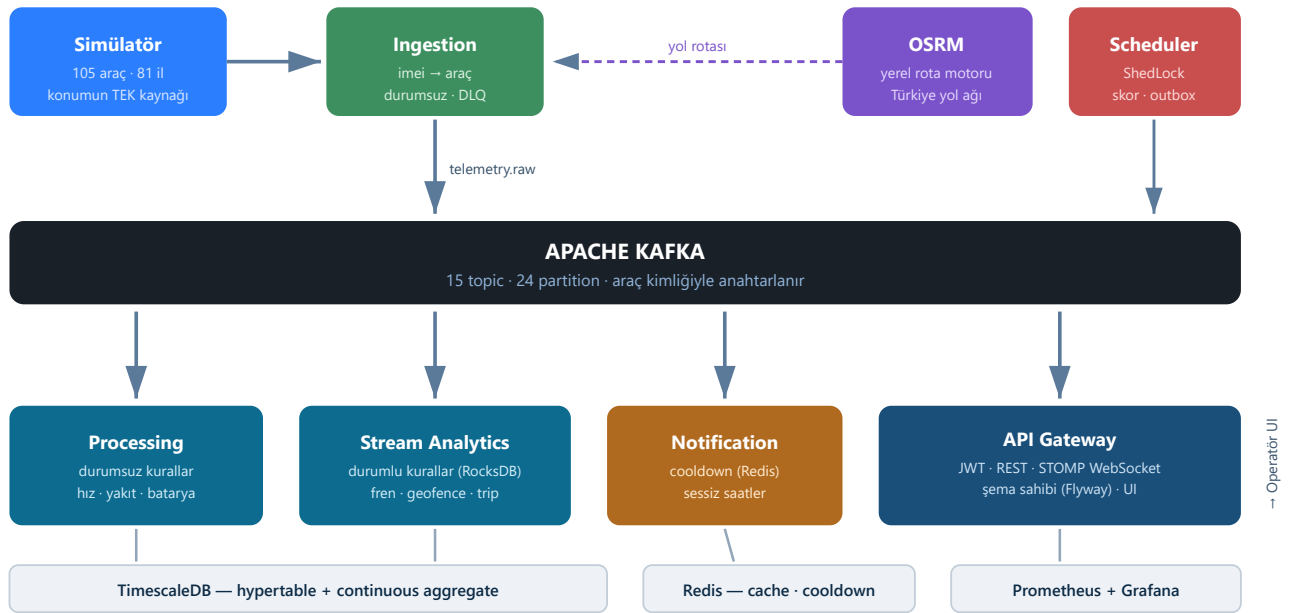
MESAJLAŞMA

Kafka 3.8.1 · 15 topic

1. Sistem neyi çözüyor?

Bir filo takip sistemi, özünde **tek yönlü bir veri hattıdır**: cihazlar konum üretir, sistem bunu anlamlandırır, operatör sonucu görür. Zorluk hacimde ve gecikmede. 105 araç saniyede bir konum gönderirse dakikada ~6.300 ölçüm eder; her ölçüm kurallardan geçer, bir kısmı ihlale döner, ihlaller bildirim döner ve tüm bunlar operatörün haritasında **bir saniyeden kısa sürede** görünmelidir.

Sistem bu yükü tek bir uygulamayla değil, **sorumluluğu bölerek** karşılar. Her servis hattın bir adımını sahiplenir ve servisler birbirini doğrudan çağırmaz — **Kafka üzerinden olaylarla** konuşur. Bunun bedeli operasyonel karmaşıklığıdır; getirisi ise her adımın bağımsız ölçeklenebilmesi ve bir adımın çökmesinin diğerlerini durdurmamasıdır.



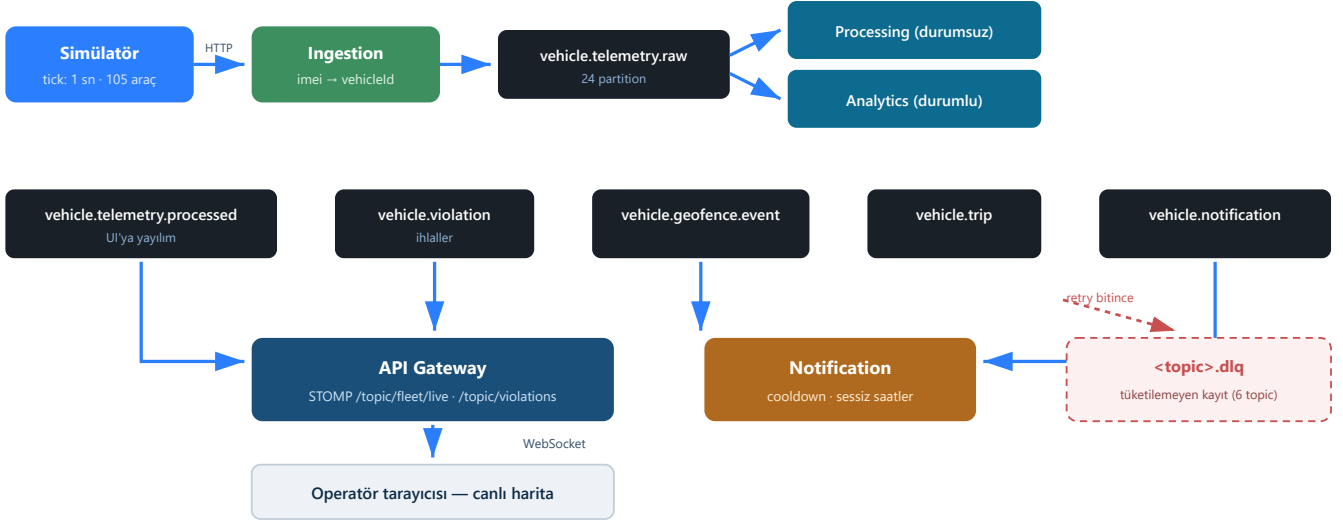
Şekil 1 — Servisler ve Kafka omurgası. Hiçbir servis diğerini doğrudan çağırmaz; tek istisna, operatör kontrolünü simülátöre ileten gateway proxy'sidir.

Modüller ve sorumlulukları

Modül	Port	Sorumluluk
vts-common	—	Olay modelleri, topic sabitleri, enum'lar, ortak Kafka tüketici desteği (deserialization + retry/DLQ politikası)
vts-simulator	8085	Filo simülatörü (Virtual Threads), OSRM yol rotaları, operatör kontrol API'si. Konumun tek kaynağı.
vts-ingestion-service	8081	Durumsuz HTTP giriş; imei→araç çözümü (Caffeine→Redis→DB), Kafka'ya yayın, DLQ
vts-processing-service	8082	Batch tüketici; JDBC batch insert, durumsuz kurallar + ihlal cooldown, outbox
vts-stream-analytics	8083	Kafka Streams; durumlu kurallar (sert fren, sürekli hız, rölanti, geofence, trip)
vts-notification-service	8084	Strategy sender'lar, cooldown (Redis), sessiz saatler
vts-api-gateway	8080	JWT güvenlik, REST, STOMP WebSocket, şema sahibi (Flyway), tek sayfa UI, operatör proxy'si
vts-scheduler-service	8086	ShedLock işleri: offline tespiti, skrolama, bakım, outbox publisher

2. Kritik akış: cihazdan haritaya

Bir konum ölçümünün hikâyesi. Bu yolun her adımı **ayrı bir topic**'tir; yani her adım bağımsız ölçeklenir ve bir adımın yavaşlaması diğerini bloklamaz — sadece kendi partition'ında birikir.



Şekil 2 — Telemetriden operatöre. Kesikli kırmızı: tüketilemeyen kayıtların ölü mektup yolu.

Neden ayrı topic'ler? Ham telemetri ile ihlal aynı topic'te olsaydı, ihlal tüketicisi saniyede 105 kayıt okuyup 99'unu atmak zorunda kalırdı. Ayrı topic, her tüketicie **yalnızca ilgilendiği hacmi** verir.

Kritik ölçek kararı: partition sayısı sabittir

Tüm araç-anahtarlı topic'ler **24 partition**'dır ve bu sayı sonradan artırılmaz. Kafka, anahtarı partition'a hash'leyerek dağıtır; sayı değişirse aynı aracın kayıtları farklı partition'lara düşer. Bu iki şeyi birden bozar: **araç başına sıralama** (bir aracın ölçümleri artık sırayla gelmez) ve **Kafka Streams durum depoları** (RocksDB'deki önceki-hız, trip durumu gibi veriler yanlış partition'da kalır). Bu yüzden sayı profilden bağımsız olarak sabitlenmiştir.

3. Araç taksonomisi ve kural motoru

Sistemin en son ve en önemli mimari düzeltmesi burada. Önceden "tip" birinci sınıf bir kavram değildi: eşikler `Kamyonlar` / `Otomobiller` adlı **gruplara** asılıydı. Grup bir organizasyon kavramıdır (depo, müşteri, bölge); tip ise aracın *ne olduğu*dur. Bu ikisi karışınca, tip bilgisine ihtiyaç duyan her yer ya grup adından tahmin ediyor ya `HELICOPTER` string'ini koda gömüyordu — ve muafiyet iki serviste **birbirinden farklı** iki kez yazılmıştı.



Boş bir kategori, bir deniz filosunun ifade edilebilir olduğunu tek bir tekne yokken söyleyebilmenin yoludur.

Şekil 3 — Kategori → tip taksonomisi. `vehicle.type` artık bir CHECK kısıtı değil, `vehicle_type` tablosuna gerçek bir yabancı anahtardır.

Hangi kural hangi tipe uygulanır?

İhlaller **kara araçlarına özeldir**; istisna, yola değil makineye ait olan yakıt/batarya kurallarıdır. Hem uygulanabilirlik hem eşik veridir — `rule_assignment` tablosunun `VEHICLE_TYPE` kapsamlı satırları. Kod değişmeden ayarlanabilir ve iki kural motoru da **aynı satırları** okur.

Kural	Otomobil	Tır	Motor	Helikopter	Değerlendiren
<code>SPEED_LIMIT</code>	110 km/s	80	90	uygulanmaz	durumsuz
<code>SUSTAINED_SPEEDING</code>	110	80	90	uygulanmaz	durumlu
<code>HARSH BRAKING</code>	-40	-30	-50	uygulanmaz	durumlu
<code>IDLING</code>	✓	✓	✓	uygulanmaz	durumlu
<code>GEOFENCE_ENTER/EXIT</code>	✓	✓	✓	uygulanmaz	durumlu
<code>LOW_FUEL</code>	%15	%20	%10	%25	durumsuz
<code>LOW_BATTERY</code>	%20	%20	%20	%20	durumsuz

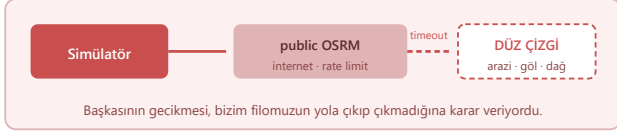
Sert frende neden küçük sayı daha sıkı? Eşik bir *yavaşlama deltasıdır* (ardışık ölçümler arası negatif km/s). Kütle, neyin şiddetli sayılacağını belirler: yüklü bir tır tek ölçümde 30 km/s kaybediyorsa bu şiddetlidir; motosiklet 50'yi rutin ve güvenle yapar. Ortak -40 eşiği, tırları geç yakalayıp motosikletleri sürekli işaretliyordu.

Doğrulama: Analytics registry açılışta **810 (kural, araç) çifti** yüklüyor — 100 kara aracı × 8 kural + 5 helikopter × 2 kural. Sayı, matrisin beklendiği gibi çözüldüğünün kendi başına kanıtı.

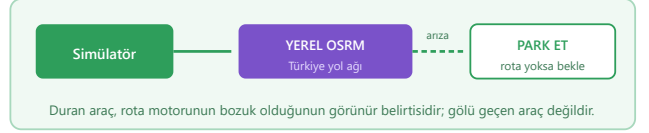
4. Araçlar yolda nasıl tutuluyor?

Kara araçlarının araziden geçmesi bir kod hatası değil, **bilinçli bir tercihin sonucuydu**. Simülatör OSRM'in *public demo sunucusuna* bağlıydı; o yavaşladığında koda gömülü fallback devreye giriyor ve araca A'dan B'ye **düz çizgi** veriyordu: araç yine gidiyor, yine varıyor, trip yine kapanıyordu — ama Türkiye'yi dağdan, gölden, tarladan keserek. Tasarım, "yolda kalmak" yerine "hareket etmeye devam etmek"i seçmişti.

ÖNCE — kırılğan



SONRA — dürist



Ölçüm — 105 aracın konumu OSRM'e soruldu: "en yakın yol kaç metre?"

100 kara aracı
100'ünün 100'ü de yolun **1 metre** içinde (en kötü 0.1 m)

5 helikopter
yollardan ortalama **450 m**, en uzak 1063 m — özgürce uçuyor

Şekil 4 — Yol takibinin öncesi/sonrası ve ölçüm sonucu. İki grubun zıtlığı, ayrımın gerçek olduğunu gösterir: kara araçları tesadüfen yolda değil.

Yerel OSRM'in bedeli

`docker compose up` ilk çalıştığında ~400 MB Türkiye OSM verisi indirir ve ön işler (`osrm-extract` → `partition` → `customize`). Bu 15–40 dakika sürer ve `osrm-extract` tek başına **~6 GB RAM** ister. Sonuç bir volume'da kalır; sonraki açılışlar saniyeler sürer ve `osrm-routed` birkaç yüz MB ile çalışır. Bedeli bir kereliktir; karşılığında sistem internetten bağımsız hale gelir.

Bilinen tuzak: Ön işleme, yığının geri kalanıyla aynı bellek havuzunu paylaşırsa `exit 137` (OOM) ile düşer. Çözüm README'de: yığını durdurup `osrm-init` 'i tek başına çalıştırmak.

Operatör kontrolü: taşıma bir kilit değil, ışınlanmadır

Operatör bir aracı haritada çift tıklayarak taşır. Kara aracı yalnızca **yol üzerine** taşınabilir; yol dışı tıklama reddedilir ve araç yerinde kalır. Eskiden tıklama sessizce en yakın yola oturtuluyordu — 26 km uzaktakine bile. Görülebilen bir ret, görülemeyen bir düzeltmeden iyidir.

Taşınan araç **durmaz**: bırakıldığı noktadan **yeni bir hedefe**, kendi seyir hızıyla yola çıkar. Önceden taşıma aracı *donduruyordu* (hız 0) ve ayrı bir "otomatiğe döndür" butonu gerekiyordu — her taşıma bir arıza gibi görünüyordu ve aracın trip'ini durduruyordu.

Hedef neden yeniden seçilir? Taşınan araç önce *eski* hedefini koruyordu ve bu, taşımayı kendi kendini iptal eden bir işleme çeviriyordu: hedef genelde aracın geldiği bölgededir, dolayısıyla Hatay'dan Ankara'ya taşınan bir tırın hâlâ Kayseri'ye varması gerekiyor ve hemen güneydoğuya — geldiği yöne — dönüyordu. Operatör aracı 700 km taşıyıp onun geri sürüşünü izliyordu. Hedef artık aracın *yeni* konumundan seçilir; "taşıma"nın yerinde kalan tek okuması budur.

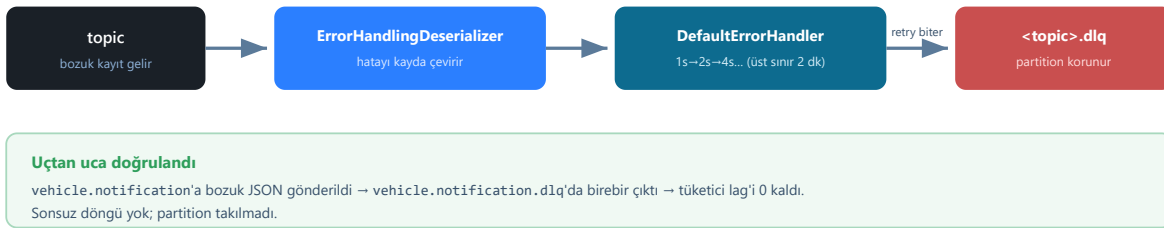
Durum	Sonuç	Neden
Yol üstü tıklama	<code>moved: true</code>	Araç oraya gider, hedefine yeni rotayla devam eder
Yol dışı tıklama	<code>OFF_ROAD</code>	En yakın yol <code>road-click-tolerance-meters</code> (50 m) dışında
Geofence aracı	<code>LOOP_VEHICLE</code>	Konumu her tick'te döngüden hesaplanır; taşıma geri alınır
Rota motoru kapalı	<code>ROUTING_UNAVAILABLE</code>	Yol üstü olduğu doğrulanamaz
Helikopter	<code>moved: true</code>	Uçar; her nokta erişilebilir

5. Dayanıklılık: hata olduğunda ne oluyor?

Zehirli mesaj (poison pill)

Bir tüketici çözemediği bir kayda takılırsa, o kaydı sonsuza kadar yeniden dener ve **arkasındaki tüm partition durur**. Sistemde bu koruma yalnızca tek bir serviste vardı; gateway ve notification listener'ları container varsayılanıyla çalışıyordu.

Daha derin bir kusur da vardı: hiçbir config `ErrorHandlingDeserializer` kullanmıyordu. Bozuk bir payload, tüketicinin *poll döngüsünde* patlar — orada hiçbir error handler onu göremez. Yani DLQ vardı ama **poison-pill'e karşı çalışmıyordu**.



Şekil 5 — Ortak retry/DLQ politikası. Artık `vts-common` 'daki tek bir auto-configuration'dan gelir; bir servis onu hatırlayarak değil, bağımlılıkla kazanır.

Tek yerde tanımlı DLQ yönlendirmesi. `Topics.dlqFor()` her topic için hedefi verir: `<topic>.dlq`. Tek istisna `vehicle.telemetry.raw` 'dur — onun ölü mektupları tarihsel olarak `vehicle.telemetry.dlq` 'ya gider. Bu istisnayı tüketicide değil çözümleyicide tutmak, **tek bir ortak error handler'ın** her topic için doğru kalmasını sağlar.

Diğer dayanıklılık kararları

Konu	Karar ve gerekçe
HTTP timeout	Her <code>RestClient</code> 'a açık connect/read timeout. Yanıt vermeyen bir simülatör, operatör isteğinin servlet thread'ini süresiz tutar; yeterince çoğu gateway'in havuzunu tüketir ve canlı haritayı da birlikte götürür.
Idempotent yazım	Telemetri <code>ON CONFLICT DO NOTHING</code> ile yazılır; yeniden teslim (at-least-once) kopya üretmez.
İhlal cooldown	Araç+kural başına bir pencere. Sürekli limit aşan bir araç, ölçüm başına değil olay başına ihlal üretir — DB/Kafka/WebSocket yükünü telemetri hızına değil olay sayısına bağlar.
Rota tick dışında	OSRM çağırısı saniyeler sürebilir; 1 saniyelik tick'i bloklamaması için rota atama arka planda yapılır.
Bellek tavanı	Her servise <code>mem_limit</code> + <code>MaxRAMPercentage</code> . JVM, limit yoksa heap tavanını host RAM'inden seçer ve 8 servis çarpınca RAM şişer.

6. Veri modeli ve okuma yolu

Şemanın tek sahibi **API gateway**'dir; açılışta Flyway ile 22 migration uygular. Diğer servisler aynı veritabanını okur/yazar ama şemayı değiştirmez — böylece migration sırası belirsiz kalmaz.

Neden TimescaleDB?

Telemetri bir **hypertable**'dir: zamana göre otomatik bölümlenir, eski bölümler sıkıştırılır ve sorgu yalnızca ilgili aralığı tarar. Üstüne **continuous aggregate** (`telemetry_1min`) kurulur: dakikalık ortalama/maksimum değerler arka planda hazırlanır.

Okuma yolu ham tabloya hiç dokunmaz. Araç geçmişi grafiği `telemetry_1min` sürekli toplamasından okunur. 3 saatlik bir grafik ham tabloda ~10.800 satır tarardı; toplamada 180 satır okur.

Katmanlama

Sorgular controller'larda değil **repository'lerde** yaşar. Çoğu tablo JPA ile modellenir; ama raporlama sorguları PostGIS ve Timescale'e özgü şeylere uzanır — `ST_AsGeoJSON` , `time_bucket` , keyset sayfalama — ki JPA bunları ifade etmez. Bu yüzden okuma yolu `JdbcTemplate` tabanlı sorgu sınıflarıdır ve satırları entity'ye değil doğrudan DTO'ya map eder: bu satır şekillerinin entity'si yoktur.

Keyset (cursor) sayfalama

İhlal listesi `OFFSET` ile değil, son satırın `(occurred_at, id)` ikilisiyle sayfanır. `OFFSET 10000` , veritabanına 10.000 satırı okuyup atmasını söyler; keyset ise doğrudan indeksin ilgili noktasına atlar — sayfa numarası büyüdükçe yavaşlamaz.

Alan	Seçim
Veritabanı	TimescaleDB (PostgreSQL 16) + PostGIS coğrafi tipler
Migration	Flyway · 22 sürüm · şema sahibi yalnızca gateway
Cache	Caffeine (süreç içi) → Redis (paylaşılan) → DB
Kimlik	JWT (HS256, Nimbus) · BCrypt · rol tabanlı metot güvenliği
Canlı yayın	STOMP over WebSocket · araç konumları throttle'lı delta olarak

7. Mimarinin özeti

Sistemi şekillendiren beş karar

Karar	Neyi satın aldı
Olay güdümlü ayrıştırma Servisler yalnızca Kafka üzerinden konuşur	Her adım bağımsız ölçeklenir; bir tüketicinin çökmesi üreticiyi durdurmaz. Bedeli: operasyonel karmaşıklık ve nihai tutarlılık.
Durumsuz/durumlu kural ayrımı Processing vs Kafka Streams	Tek ölçüme bakan kurallar ucuz ve yatay ölçeklenir; pencere/geçmiş isteyenler RocksDB'li topolojide kalır.
Sabit partition sayısı (24)	Araç başına sıralama ve durum depolarının tutarlılığı korunur — sonradan artırmak ikisini de bozardı.
Taksonomi birinci sınıf kategori → tip → kural	"Bu bir helikopter mi" sorusu koddan kalktı; muafiyet tek bir veri kaynağında toplandı ve iki motor artık ayrışamaz.
Yerel rota motoru	Kara araçlarının yolda kalması, başkasının sunucusunun gecikmesine bağlı olmaktan çıktı.

Tekrar eden bir tema: sessiz düzeltme yerine görülebilir ret

Sistemin son turdaki düzeltmelerinin çoğu aynı ilkeye indi. Bir işlem yapılamıyorsa, sistem **onu yapmış gibi davranmamalı**:

- Yol rotası yoksa araç **park eder** — düz çizgiyle gölü geçmez.
- Yol dışı tıklama **reddedilir** — sessizce 26 km öteye oturtulmaz.
- Taşınamayan geofence aracı "**taşınamaz**" **der** — `moved:true` deyip yerinde kalmaz.
- Çözülemeyen kayıt **DLQ'ya düşer** — sonsuza kadar yeniden denenip partition'ı kilitlemez.

Bu tercihlerin ortak bedeli, sistemin bazen "daha az iş yapıyor" görünmesidir: duran bir araç, reddedilen bir tıklama. Karşılığında ise gördüğünüz her şey **gerçektir** — ve bir şey bozulduğunda bunu gizlenmiş bir yanlış davranıştan değil, doğrudan belirtiden anlarsınız.

Sistemin ölçülen durumu — 17 konteyner ayakta, hiçbirinde OOM veya restart yok. Telemetry son 60 saniyede 6.292 ölçüm / 105 araç. 104/104 araç yol rotası aldı. İhlaller yalnızca kara araçlarında. 34 test geçiyor.